# Übungsblatt 6: Multicore, Speicherbesitz und Cache

**Echtzeitsysteme (EZS)**

Prof. Dr.-Ing. Daniel Klünder

## Aufgabe 1: Zuständigkeit und gemeinsames Zeitbudget (8 Punkte)

Eine Messstation betreibt einen FreeRTOS-Kernel auf M4 und eine eigenständige M7-Hauptschleife.

1. Ordnen Sie AMP oder SMP zu. Darf M7 ein im SRAM abgelegtes M4-Mutexhandle direkt verwenden? Warum nicht? (3 Punkte)
2. Eine Antwort benötigt disjunkt 3 ms eigene Arbeit, 4 ms Warten auf die Gegenstelle und 1 ms Businterferenz. Prüfen Sie $D = 7\,\text{ms}$. (2 Punkte)
3. Zwei Kerne ändern unterschiedliche Felder desselben GPIO-Konfigurationsregisters jeweils per Read-Modify-Write. Warum verhindert getrennte Pinzuordnung den Konflikt nicht? Formulieren Sie eine Besitzerregel. (3 Punkte)

## Aufgabe 2: Einen neuen Shared-Entwurf prüfen (8 Punkte)

Nur für diesen Papierentwurf: vier natürlich auf vier Byte ausgerichtete `uint32_t`-Felder `magic`, `sequence`, `command`, `value`. Das Objekt liegt auf einer 32-Byte-Grenze in einer auf 32 Byte aufgefüllten Sektion. Reserviert ist eine 32-KiB-MPU-Region ab `0x38008000`.

1. Geben Sie Feldoffsets, Typgröße, Sektionsreserve, letzte Regionsadresse und SIZE-Code an; es gilt Größe $= 2^{\text{SIZE}+1}$ Byte. (4 Punkte)
2. M7-Map: Objekt bei `0x38008000`; M4-Map: `0x38008040`. Genügt derselbe C-Header? Was ist zu korrigieren? (2 Punkte)
3. `NOLOAD` ist gesetzt, M4 nullt nach dem Boot aber einen überlappenden Bereich. Warum bleibt das ein Fehler? Wer initialisiert wann, und reicht eine alte Kennung nach einseitigem Reset? (2 Punkte)

## Aufgabe 3: Cachekopien sind keine gemeinsame Sicht (8 Punkte)

1. M7 hat nach einem Store eine Dirty-Kopie mit 27; SRAM enthält noch 19. Was kann M4 lesen? Umgekehrt schreibt M4 44 in SRAM, M7 hält eine gültige alte Kopie mit 27. Was kann M7 lesen? Reichen `volatile` und DMB? (3 Punkte)
2. Bei 32-Byte-Cachezeilen beginnt ein 12-Byte-Puffer bei `0x3800003C`. Bestimmen Sie den vollständig zu wartenden Bereich, Länge und Zeilenzahl. (2 Punkte)
3. M7 ändert Wort A seiner Dirty-Zeile auf 41, B bleibt dort null. M4 schreibt inzwischen B im SRAM auf 73. Was kann ein Clean der M7-Zeile bewirken? Welche Besitzgranularität ist nötig? (3 Punkte)

## Aufgabe 4: Veröffentlichen, konsumieren und abbrechen (8 Punkte)

Ein Ein-Platz-Kanal wird phasenweise übergeben. Bei REQ besitzt M4 den Wert, bei RESP M7. M7 bietet nacheinander 4, 11 und 23 an; M4 berechnet $y = 3x + 2$ und publiziert eine Antwort. Es gibt genau einen Teilnehmer je Kern, geordneten nicht cachebaren Speicher und ausgerichtete Wortzugriffe als Plattformannahme.

1. Bestimmen Sie Antworten und Reihenfolge von Nutzwert, DMB und Phasenflag beim Publizieren und Konsumieren. (4 Punkte)
2. Eine niedrigere M7-MPU-Region ist nicht cachebar, eine passende höher nummerierte Region cachebar. Welches Attribut gilt? Macht Shareable daraus Hardware-Kohärenz? (2 Punkte)
3. Ein lokaler Timeout tritt ein. Darf der wartende Kern den Kanal sofort nullen? Was setzt ein Neustart voraus? (2 Punkte)

## Aufgabe 5: Sichtbarkeit und Atomarität trennen

Ein nicht cachebarer Zähler steht bei 62. Beide Kerne lesen 62, erhöhen privat um eins und schreiben 63. War die MPU-Einstellung deshalb wirkungslos? Warum hilft auch ein nur vom M4 verwendeter Mutex nicht gegen M7?

## Aufgabe 6: Eine eigene Antwortfunktion für den zweiten Kern schreiben

**Programmieren, etwa 25–35 Minuten.** Die vorbereitete Dual-Core-Basis `code/lab06/` hält Bootfolge, beide Linkerlayouts und den nicht cachebaren Shared-Bereich fest. In beiden Zielen ist `LAB06_STUDENT_EXERCISE` definiert. M7 sendet 4, 11 und 23. Sie implementieren auf CM4 `Lab06_Respond()` in `respond_student.c`; die Basis ruft sie erst bei beobachtetem `REQUEST_M4` auf.

1. Übernehmen Sie nach einer DMB-Barriere `ipc.payload` in eine private Variable. Berechnen Sie $3x + 2$, schreiben Sie das Ergebnis und veröffentlichen Sie anschließend mit korrekter Reihenfolge `RESPONSE_M7`. Der Schritt führt weder einen Kernel-Warteaufruf noch eine eigene Warteschleife aus.
2. Zeichnen Sie vor dem Build die drei erwarteten Antworten und die jeweilige Schreibberechtigung auf. Die C-Basis prüft Antworten, Phasen und Timeouts.
3. Bauen und laden Sie beide Images. Prüfen Sie `lab06_cm7_reply`, beide Done-Kennzeichen und `lab06_error`. Lesen Sie gemeinsame Diagnosen nur bei angehaltenen beiden Kernen. Die Startdatei ohne Implementierung endet absichtlich im Protokollfehler und zählt keine gültige Antwort.
4. Ergänzen Sie nur nach erfolgreichem Gesamtlauf die vorbereitete Anzeige auf **LD1/PB0**. Erklären Sie, weshalb eine leuchtende LED keine Cache- oder Fristgarantie ist. Ein cachebarer Fehlerbuild ist für diese Aufgabe nicht nötig.

**Vorbereitete Basis:** Projektanleitung, STM32CubeMX und VS Code mit STM32 VS Code Extension für **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Board und USB/ST-LINK genügen. Eigene Implementierung und tatsächlichen Prüfstatus festhalten; Debuggerdiagnosen bei angehaltenem zuständigem Kern lesen.

**Quellen:** Buttazzo, *Hard Real-Time Computing Systems*; ST UM2408 und RM0399; Arm CMSIS und gegebenenfalls CMSIS-RTOS2. Papierwerte sind Lehrannahmen.